

Windows 开发工具链安装文档

pixi · Julia / juliaup — 基础安装 & 可用性验证

第一部分：pixi 安装

pixi 是基于 Conda 生态构建的跨平台包管理器，本体为单一可执行文件（`pixi.exe`），无需管理员权限即可安装。

1.1 最简安装（推荐，需网络）

在 PowerShell 中执行：

```
1 powershell -ExecutionPolicy Bypass -c "irm -useb https://pixi.sh/install.ps1 | iex"
```

执行完成后，`pixi.exe` 被安装到 `%UserProfile%\pixi\bin` 并自动写入用户 PATH。重启终端后生效。

1.2 最可靠安装（手动下载安装包）

第一步：下载安装包

文件	下载地址
下载页面（始终最新）	https://github.com/prefix-dev/pixi/releases/latest
x64 ZIP（推荐）	https://github.com/prefix-dev/pixi/releases/latest/download/pixi-x86_64-pc-windows-msvc.zip
x64 MSI	https://github.com/prefix-dev/pixi/releases/latest/download/pixi-x86_64-pc-windows-msvc.msi
ARM64 ZIP	https://github.com/prefix-dev/pixi/releases/latest/download/pixi-aarch64-pc-windows-msvc.zip

不确定架构时，在 PowerShell 执行 `$env:PROCESSOR_ARCHITECTURE` 查看，现代 PC 通常为 `AMD64`（即 x64）。

第二步 A：ZIP 方案（手动部署）

解压 ZIP 得到 `pixi.exe`，在 PowerShell 中执行：

```
1 # 创建目标目录（路径可自定义）
2 New-Item -ItemType Directory -Force -Path "C:\tools\pixi"
3
4 # 将 pixi.exe 复制到目标目录
5 Copy-Item "<下载路径>\pixi.exe" "C:\tools\pixi\pixi.exe"
6
7 # 永久写入用户 PATH（无需管理员权限）
8 $p = [Environment]::GetEnvironmentVariable("PATH", "User")
9 [Environment]::SetEnvironmentVariable("PATH", "$p;C:\tools\pixi", "User")
```

重启终端后生效。

第二步 B: MSI 方案

双击 `.msi` 文件，按引导安装即可，安装程序自动配置 PATH。

1.3 可用性测试

前置条件：重启终端后再执行。

测试一：版本与路径

```
1 pixi --version
2 where.exe pixi
```

预期输出：

```
1 pixi 0.x.x
2 C:\Users\<用户名>\.pixi\bin\pixi.exe
```

测试二：创建临时项目并安装包（需网络）

```
1 mkdir $env:TEMP\pixi-test; cd $env:TEMP\pixi-test
2 pixi init .
3 pixi add python
4 pixi run python --version
5
6 # 清理
7 cd $env:TEMP; Remove-Item -Recurse -Force pixi-test
```

若 `python --version` 正常输出版本号，则 pixi 工具链完整可用。

第二部分：Julia 安装（通过 juliaup）

Julia 官方推荐通过 juliaup 安装。juliaup 是 Julia 的版本管理器，负责下载 Julia 本体、管理多版本并维护 PATH。

2.1 最简安装（推荐，需网络）

在 PowerShell 中执行（winget 为 Windows 10 2004 及以上内置）：

```
1 | winget install --id JuliaLang.Juliaup
```

执行后重启终端，`julia` 和 `juliaup` 命令均可用。

2.2 最可靠安装（手动下载安装包）

提供两种安装包，均为官方发布，无需访问微软商店。

方案 A：MSIX App Installer（推荐）

文件	下载地址
<code>Julia.appinstaller</code>	https://install.julialang.org/Julia.appinstaller

下载后双击运行，按引导完成安装，自动配置 PATH。

也可在 PowerShell 中执行：

```
1 | Add-AppxPackage -AppInstallerFile .\Julia.appinstaller
```

方案 B：MSI 安装包（备选）

注意：MSI 方案不含 `juliaup` 自动更新功能，仅在 MSIX 方式不可用时使用。安装完成后需从开始菜单启动一次 Julia 以触发首次初始化。

文件	下载地址
<code>Julia-x64.msi</code> （64位，推荐）	https://install.julialang.org/Julia-x64.msi
<code>Julia-x86.msi</code> （32位）	https://install.julialang.org/Julia-x86.msi

双击 `.msi` 文件，按引导安装，默认为当前用户安装，无需管理员权限。

2.3 可用性测试

前置条件：重启终端后再执行。MSI 方案需先从开始菜单启动一次 Julia。

测试一：juliaup 状态

```
1 | juliaup status
```

预期输出：

```
1 | Default Channel Version
2 | -----
3 | * release Julia 1.x.x/release
```

测试二：Julia 基本运行

```
1 | julia --version
2 | julia -e "println(1 + 1)"
3 | julia -e "versioninfo()"
```

预期输出：

```
1 | julia version 1.x.x
2 | 2
3 | Julia Version 1.x.x ...
```

测试三：包管理器可用性（需网络）

```
1 | julia -e "using Pkg; Pkg.activate(temp=true); Pkg.add(\"Example\"); using Example;
  | println(\"Pkg OK\")"
```

若输出 `Pkg OK`，则 Julia 包管理器完整可用。